exp3.py

```
# STEP 1: Select a Real-World Dataset
# ==============================================================================
# STEP 2: Import Required Libraries
# ==============================================================================
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats
import statsmodels.api as sm
# ==============================================================================
# STEP 3: Load Dataset & Calculate Descriptive Statistics
# ==============================================================================
# CSV file load karein
try:
    df = pd.read_csv('WA_Fn-UseC_-HR-Employee-Attrition.csv')
    print("Dataset successfully loaded!")
except FileNotFoundError:
    print("File not found! Generating dummy dataset matching IBM HR structure for demonstration...")
    np.random.seed(42)
    n = 200
    df = pd.DataFrame({
        'Attrition': np.random.choice(['Yes', 'No'], size=n, p=[0.2, 0.8]),
        'MonthlyIncome': np.random.randint(2000, 20000, size=n),
        'TotalWorkingYears': np.random.randint(1, 30, size=n),
        'JobSatisfaction': np.random.choice([1, 2, 3, 4], size=n),
        'JobRole': np.random.choice(['Sales Executive', 'Research Scientist', 'Laboratory Technician'], size=n)
    })

# Descriptive Statistics (Mean, Median, Variance, Standard Deviation)
numerical_features = ['MonthlyIncome', 'TotalWorkingYears', 'JobSatisfaction']

print("\n--- DESCRIPTIVE STATISTICS ---")
print("MEAN:\n", df[numerical_features].mean())
print("\nMEDIAN:\n", df[numerical_features].median())
print("\nVARIANCE:\n", df[numerical_features].var())
print("\nSTANDARD DEVIATION:\n", df[numerical_features].std())

# ==============================================================================
# STEP 4: Pearson Correlation Matrix & Visualization
# ==============================================================================
print("\n--- PEARSON CORRELATION MATRIX ---")
corr_matrix = df[numerical_features].corr(method='pearson')
print(corr_matrix)

# Visualization
plt.figure(figsize=(8, 5))
sns.heatmap(corr_matrix, annot=True, cmap='coolwarm', fmt=".2f", vmin=-1, vmax=1)
plt.title('Pearson Correlation Matrix')
plt.tight_layout()
plt.show()

# ==============================================================================
# STEP 5 & 6: Formulate Hypotheses & Perform Independent Sample t-test
# ==============================================================================
# H0: Monthly Income me Attrition (Yes vs No) ke beech koi significant difference nahi hai.
# H1: Monthly Income me Attrition (Yes vs No) ke beech significant difference hai.

income_attrition_yes = df[df['Attrition'] == 'Yes']['MonthlyIncome']
income_attrition_no = df[df['Attrition'] == 'No']['MonthlyIncome']

t_stat, p_val_ttest = stats.ttest_ind(income_attrition_yes, income_attrition_no, equal_var=False)

print("\n--- INDEPENDENT SAMPLE T-TEST RESULTS ---")
print(f"t-statistic: {t_stat:.4f}")
print(f"p-value: {p_val_ttest:.4e}")

if p_val_ttest < 0.05:
    print("Decision: Reject Null Hypothesis (H0). Significant difference exists in monthly income between employees who leav
else:
    print("Decision: Fail to reject Null Hypothesis (H0). No statistically significant difference found.")

# ==============================================================================
# STEP 7: One-Way ANOVA Test
# ==============================================================================
# Check if JobSatisfaction varies significantly across JobRoles
roles = df['JobRole'].unique()
```

```
role_groups = [df[df['JobRole'] == role]['JobSatisfaction'] for role in roles]

f_stat, p_val_anova = stats.f_oneway(*role_groups)

print("\n--- ONE-WAY ANOVA RESULTS ---")
print(f"F-statistic: {f_stat:.4f}")
print(f"p-value: {p_val_anova:.4e}")

if p_val_anova < 0.05:
    print("Decision: Reject Null Hypothesis (H0). Job satisfaction differs significantly across job roles.")
else:
    print("Decision: Fail to reject Null Hypothesis (H0). No significant difference in job satisfaction across job roles.")

# ==============================================================================
# STEP 8 & 9: Simple Linear Regression & Interpretation
# ==============================================================================
# Predictor (X): TotalWorkingYears, Target (y): MonthlyIncome
X = df['TotalWorkingYears']
y = df['MonthlyIncome']

# Constant add karna regression equation ke liye (y = mx + c)
X_with_const = sm.add_constant(X)

model = sm.OLS(y, X_with_const).fit()

print("\n--- SIMPLE LINEAR REGRESSION SUMMARY ---")
print(model.summary())

# Outputs Interpret karna
print("\n--- REGRESSION METRICS INTERPRETATION ---")
print(f"R-squared Score: {model.rsquared:.4f}")
print(f"Intercept (c): {model.params['const']:.4f}")
print(f"Slope Coefficient (m): {model.params['TotalWorkingYears']:.4f}")
print(f"P-value of Predictor: {model.pvalues['TotalWorkingYears']:.4e}")
```